

MyMquid

Backend API Specification

Complete API reference for the MyMquid Elevate platform backend.
Designed for NestJS implementation. Covers authentication, blog
management, dashboard analytics, notifications and user profile.

Version 1.0

May 2026

NestJS + PostgreSQL

JWT Auth

REST

Table of Contents

01 — OVERVIEW

Architecture & Tech Stack	3
Base URL & Versioning	3
Authentication Mechanism	3
Role-Based Access Control	4
Error Response Format	4
Pagination	4

02 — DATA MODELS

AdminUser	5
BlogPost	5
DashboardStats	6
ActivityEvent	6
ChartDataPoint	6
Notification	6

03 — AUTHENTICATION API

POST /auth/login	7
POST /auth/logout	7
POST /auth/forgot-password	8
POST /auth/reset-password	8
GET /auth/me	8

04 — BLOG API

GET /blog	9
POST /blog	10
GET /blog/:id	11
PUT /blog/:id	11
DELETE /blog/:id	12

05 — DASHBOARD API

GET	/dashboard/stats	13
GET	/dashboard/activity	13
GET	/dashboard/chart	14

06 — NOTIFICATIONS API

GET	/notifications	14
PATCH	/notifications/:id/read	15
PATCH	/notifications/read-all	15

07 — PROFILE API

GET	/profile	15
PUT	/profile	16
PUT	/profile/password	16

08 — FRONTEND INTEGRATION NOTES

Mock-to-Real Swap Point	17
Axios Instance Setup	17
Categories	17
Public Blog vs Admin Blog	17



01 — Overview

Architecture, authentication, conventions and error handling

Architecture & Tech Stack

BACKEND

Framework	NestJS (Node.js)
Language	TypeScript
Database	PostgreSQL (recommended)
ORM	TypeORM or Prisma
Auth	JWT (access + refresh tokens)
Validation	class-validator + class-transformer

FRONTEND (EXISTING)

Framework	React 19 + Vite 7
Language	TypeScript 5.8
State	Zustand (persist to localStorage)
HTTP Client	Axios (to be set up)
Rich Text	Tiptap (JSON format)
Auth Storage	localStorage via Zustand persist

Base URL & Versioning

URL STRUCTURE

Development:	<code>http://localhost:3000/api/v1</code>
Production:	<code>https://api.mymquid.com/api/v1</code>

REQUEST HEADERS

<code>Content-Type: application/json</code>
<code>Accept: application/json</code>
<code>Authorization: Bearer <access_token></code>

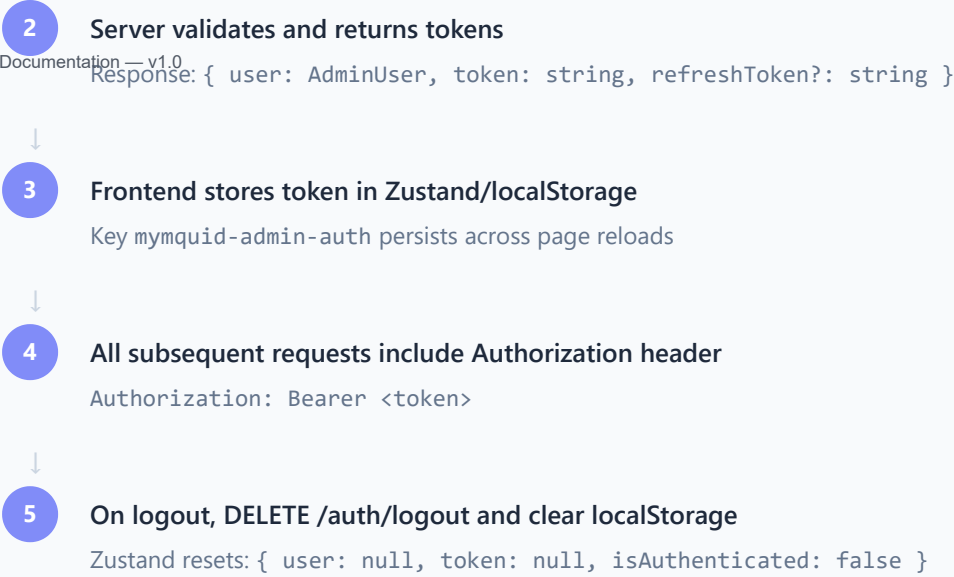
All endpoints are prefixed with `/api/v1`. The frontend's mock API layer is in `src/admin/mock/api.ts` — that file is the single swap point where mock calls will be replaced with real Axios requests once the backend is live.

Authentication Mechanism

The frontend stores token and user in `localStorage` via `Zustand persist` (key: `mymquid-admin-auth`). On login, the token is stored and attached to all subsequent requests via an `Axios` interceptor.

AUTHENTICATION FLOW

- 1 Client POSTs credentials
- POST `/auth/login` with `{ email, password }`



Role-Based Access Control (RBAC)

The frontend already enforces RBAC in the UI — the backend must enforce it at the API layer. All admin routes require a valid JWT. Delete operations require `super_admin`.

ROLE	DESCRIPTION	PERMISSIONS
super_admin	Full access	Create, read, update, delete posts; manage all settings; view all dashboard data
staff	Content author	Create, read, update own posts; cannot delete posts; view dashboard

Error Response Format

STANDARD ERROR SHAPE

```
{
  "statusCode": 400,
  "message": "Validation failed",
  "errors": [
    {
      "field": "email",
      "message": "must be a valid email"
    }
  ]
}
```

HTTP STATUS CODES USED

200	Success (GET, PUT, PATCH)
201	Created (POST)
204	No Content (DELETE)
400	Validation error
401	Unauthenticated
403	Forbidden (wrong role)
404	Resource not found
409	Conflict (duplicate slug)
500	Internal server error

Pagination

QUERY PARAMETERS (ALL LIST ENDPOINTS)

PARAM	TYPE	DEFAULT	DESCRIPTION
page	number	1	Page number (1-indexed)
limit	number	10	Items per page (max 100)

PAGINATED RESPONSE SHAPE

```
{
  "data": [...],
  "total": 42,
  "page": 1,
  "limit": 10,
  "totalPages": 5
}
```

The frontend blog list uses client-side pagination over the full list today. When swapped to the real API, pass page and limit from the UI's pagination state. The frontend currently fetches 10 posts per page by default.



02 — Data Models

TypeScript types used by the frontend — mirror these exactly in your NestJS entities and DTOs

AdminUser

Returned on login and from /auth/me

TYPESCRIPT DEFINITION

```
type AdminRole = "super_admin" | "staff";

type AdminUser = {
  id: string;
  name: string;
  email: string;
  role: AdminRole;
  avatar?: string; // URL to avatar image
};
```

EXAMPLE JSON

```
{
  "id": "1",
  "name": "Patrick Evra",
  "email": "admin@mymquid.com",
  "role": "super_admin",
  "avatar": null
}
```

BlogPost

Core entity — used throughout Blog API

TYPESCRIPT DEFINITION

```
type BlogStatus =
  "draft" | "published" | "scheduled";

type BlogPost = {
  id: string;
  title: string;
  slug: string; // URL-safe, unique
  content: string; // Tiptap JSON (stringified)
  status: BlogStatus;
  category: string;
  tags: string[];
  featuredImage?: string; // URL
  seo: {
    metaTitle: string;
    metaDescription: string;
    ogImage?: string; // URL
  };
  scheduledAt?: string; // ISO 8601 (if scheduled)
  createdAt: string; // ISO 8601
  updatedAt: string; // ISO 8601
  author: {
    id: string;
    name: string;
  };
};
```

EXAMPLE JSON (TRUNCATED)

```
{
  "id": "1",
  "title": "Cloud Security",
  "slug": "cloud-security",
  "content": "{\"type\":\"doc\",...}",
  "status": "published",
  "category": "Solutions",
  "tags": ["cloud","security"],
  "featuredImage": null,
  "seo": {
    "metaTitle": "Cloud Security",
    "metaDescription": "..."
  },
  "scheduledAt": null,
  "createdAt": "2026-05-08T11:00",
  "updatedAt": "2026-05-08T11:00",
  "author": {
    "id": "1",
    "name": "Patrick Evra"
  }
}
```

Important: content is a **JSON string** (JSON editor output serialised with

FIELD CONSTRAINTS

FIELD	REQUIRED	CONSTRAINTS
title	Required	max 200 chars, non-empty
slug	Required	unique, lowercase, hyphens only (/^[a-z0-9]+(?:-[a-z0-9]+)*\$/)
content	Required	valid Tiptap JSON string, non-empty
status	Required	one of: draft , published , scheduled
category	Required	must match a valid category (see §08)
tags	Optional	array of strings, max 10 tags
seo.metaTitle	Required	max 60 chars
seo.metaDescription	Required	max 160 chars
scheduledAt	Optional	required when status === "scheduled" , must be future date

DashboardStats

```
type DashboardStats = {
  totalPosts: number;
  published: number;
  drafts: number;
  scheduled: number;
};
```

ChartDataPoint

```
type ChartDataPoint = {
  date: string; // "YYYY-MM-DD"
  posts: number; // posts created that d
};
```

ActivityEvent

```
type ActivityEvent = {
  id: string;
  type: "publish" | "draft" |
        "login" | "delete" | "edit";
  message: string; // human-readable
  time: string; // relative ("2h ago")
  createdAt: string; // ISO 8601
};
```

Note: The time field is a pre-formatted relative string. The frontend displays it as-is. You may return the raw createdAt and let the

Notification

```
type Notification = {
  id: string;
  title: string;
  message: string;
  type: "info" | "success" |
        "warning" | "error";
  read: boolean;
  createdAt: string; // ISO 8601
};
```


frontend compute relative time — update the `RecentActivity` component accordingly if you prefer.



03 — Authentication API

Login, logout, password reset, and token refresh

POST**/auth/login**

No auth required

REQUEST BODY

```
{
  "email": "admin@mymquid.com", // required
  "password": "your_password" // required
}
```

VALIDATION RULES

email valid email format, non-empty

password min 8 chars, non-empty

RESPONSE 200 OK

```
{
  "user": {
    "id": "1",
    "name": "Patrick Evra",
    "email": "admin@mymquid.com",
    "role": "super_admin"
  },
  "token": "eyJhbGciOiJIUzI1..."
}
```

ERROR RESPONSES

400 Validation error (missing fields)

401 Invalid email or password

The frontend's Zustand auth store (`src/admin/auth/useAuthStore.ts`) stores the returned user and token in `localStorage`. The token field name must remain exactly `"token"` to avoid breaking the frontend.

POST**/auth/logout****Auth Required**

REQUEST

No request body. Include `Authorization: Bearer <token>` header.

RESPONSE 204 NO CONTENT

Empty body. Invalidate the JWT on the server side (blacklist or short-lived token).

POST**/auth/forgot-password**

No auth required

REQUEST BODY

```
{
  "email": "admin@mymquid.com"
}
```

RESPONSE 200 OK

```
{
  "message": "Reset email sent if account exists"
}
```

POST

/auth/reset-password

No auth required

REQUEST BODY

```
{
  "token":      "reset_token_from_email",
  "newPassword": "new_secure_password",
  "confirmPassword": "new_secure_password"
}
```

RESPONSES

200	Password reset successfully
400	Passwords don't match
400	Token invalid or expired

GET

/auth/me

Auth Required

REQUEST

No body. JWT in Authorization header.

Used on app load to verify the stored token is still valid and refresh the user object.

RESPONSE 200 OK

```
{
  "id":      "1",
  "name":    "Patrick Evra",
  "email":   "admin@mymquid.com",
  "role":    "super_admin"
}
```



04 — Blog API

Full CRUD for blog posts — the primary feature of the admin dashboard

All Blog API endpoints require a valid JWT except **GET /blog/public** which is used by the public-facing website and is unauthenticated.

GET

/blog

Auth Required

Admin list with filters

QUERY PARAMETERS

PARAMETER	TYPE	REQUIRED	DESCRIPTION
status	string	Optional	Filter by status: <code>draft</code> <code>published</code> <code>scheduled</code> . Omit for all.
category	string	Optional	Filter by category name (exact match, URL-encoded)
search	string	Optional	Case-insensitive title search (ILIKE <code>%search%</code>)
page	number	Optional	Page number, default 1
limit	number	Optional	Items per page, default 10, max 100

RESPONSE 200 OK

```
{
  "data": [/* BlogPost[] - full objects */],
  "total": 10,
  "page": 1,
  "limit": 10,
  "totalPages": 1
}
```

Frontend behaviour: The admin Blog List page calls this with `status`, `category`, and `search` from the filter bar state. Empty string values should be treated the same as omitted (no filter applied).

POST

/blog

Auth Required

All Roles

Create new post

REQUEST BODY (CREATEBLOGPOSTDTO)

```
{
  "title": "Cloud Security Best Practices",
  "slug": "cloud-security-best-practices",
  "content": "{\"type\":\"doc\",\"content\":[...]}",
  "status": "draft",
  "category": "Solutions",
}
```

```
"tags": ["cloud", "security"],
"featuredImage": null,
"seo": {
  "metaTitle": "Cloud Security Best Practices",
  "metaDescription": "Protect your cloud workloads.",
  "ogImage": null
},
"scheduledAt": null // required if status === "scheduled"
}
```

SERVER RESPONSIBILITIES

- Generate UUID for `id`
- Set `createdAt` and `updatedAt` to `now()`
- Set `author` from the JWT payload (authenticated user)
- Validate `slug` uniqueness — return 409 if conflict
- Validate `scheduledAt` is future if `status === "scheduled"`

RESPONSE 201 CREATED

Returns the full `BlogPost` object including server-generated `id`, `createdAt`, `updatedAt`, and `author`.

GET `/blog/:id`

Auth Required Get single post by ID

PATH PARAMETER

`:id` Post UUID

RESPONSES

200 Full `BlogPost` object

404 Post not found

Used by the admin editor when navigating to `/admin/blog/edit/:id` to pre-populate the form.

PUT `/blog/:id`

Auth Required All Roles Full update

REQUEST BODY (UPDATEBLOGPOSTDTO)

Same shape as `CreateBlogPostDto`. All fields sent — frontend sends the complete object.

createdAt must not change. The frontend sends the original `createdAt` in the body; the server should ignore it and use the stored value — or simply not accept `createdAt` in the DTO. Only `updatedAt` is updated (to `now()`).

RESPONSES

200 Updated `BlogPost` object

403 Staff trying to edit another author's post

404 Post not found

409 Slug conflict with another post

Staff restriction: staff role can only update posts where `author.id` matches their own user ID.

PATH PARAMETER

:id

Post UUID

RESPONSES

204

Deleted — no body

403

Caller is not `super_admin`

404

Post not found

Role enforcement required: The frontend hides the delete button from `staff` users but the backend **MUST** also enforce this at the API level. Return `403 Forbidden` if a `staff` JWT is used.

GET

/blog/public

No auth — public website facing

QUERY PARAMETERS

PARAMETER	TYPE	DEFAULT	DESCRIPTION
<code>page</code>	number	1	Page number
<code>limit</code>	number	20	Items per page

BEHAVIOUR

- Returns only `status === "published"` posts
- Sorted by `createdAt` DESC (newest first)
- No authentication required — CORS must allow the frontend origin
- Response shape is identical to `GET /blog` (paginated)

Frontend integration point: `src/pages/Blog.tsx` currently calls `blogApi.getAll({ status: "published" })` from the mock. When swapping, replace this with an unauthenticated `GET /api/v1/blog/public` call.



05 — Dashboard API

Stats, activity feed and chart data — all require authentication

GET /dashboard/stats

Auth Required

REQUEST

No parameters.

RESPONSE 200 OK

```
{
  "totalPosts": 10,
  "published": 5,
  "drafts": 3,
  "scheduled": 2
}
```

These counts are shown as stat cards at the top of the admin dashboard. Compute them with `COUNT(*) WHERE status = ?` queries. The frontend calls this on every dashboard page load, so it should be fast — consider caching for 30–60 seconds.

GET /dashboard/activity

Auth Required

RESPONSE 200 OK

```
[
  {
    "id": "1",
    "type": "publish",
    "message": "Post published: Cloud Security",
    "time": "2h ago",
    "createdAt": "2026-05-19T08:00:00Z"
  },
  ...
]
```

ACTIVITY EVENT TYPES

TYPE	TRIGGERED BY
publish	Post status set to "published"
draft	Post saved as draft
edit	Existing post updated
delete	Post deleted
login	Admin user logged in

Returns the 20 most recent events, sorted by `createdAt DESC`. Consider an `activity_log` table populated by NestJS service layer events.

GET /dashboard/chart

Auth Required

QUERY PARAMETERS

PARAM	DEFAULT	DESCRIPTION
<code>days</code>	30	Number of days to return (max 90)

RESPONSE 200 OK

```
[
  { "date": "2026-04-20", "posts": 2 },
  { "date": "2026-04-21", "posts": 0 },
  ...
]
```

- Always return exactly `days` entries
- Fill gaps (days with 0 posts) with `{ date, posts: 0 }`
- Use `DATE(created_at)` grouping in SQL
- Sorted ascending by date
- Used to render a line/bar chart on the dashboard



06 — Notifications API

In-app notification feed — all endpoints require authentication

GET

`/notifications`

Auth Required

RESPONSE 200 OK

```
[
  {
    "id": "1",
    "title": "Post Published",
    "message": "Welcome to MyMquid is now live.",
    "type": "success",
    "read": false,
    "createdAt": "2026-05-19T08:00:00Z"
  }
]
```

NOTES

- Returns notifications for the authenticated user
- Sorted by `createdAt` DESC
- Unread count is derived by the frontend: `filter(n => !n.read).length`
- The TopNav badge displays this unread count
- Limit to 50 most recent

PATCH

`/notifications/:id/read`

Auth

PATH PARAMETER

`:id` Notification UUID

RESPONSE 200 OK

```
{
  "id": "1",
  "read": true
}
```

PATCH

`/notifications/read-all`

Auth

REQUEST / RESPONSE

No body. Marks all notifications for the authenticated user as read.

RESPONSE 200 OK

```
{
  "updated": 3
}
```


Route order matters: Register `/read-all` before `/:id/read` in NestJS to avoid the literal string "read-all" being treated as an `:id` param.



07 — Profile API

Authenticated user's own profile — name, email, avatar, password

GET /profile

Auth Required

Returns the authenticated user's own profile. Identical shape to AdminUser.

RESPONSE 200 OK

```
{
  "id": "1",
  "name": "Patrick Evra",
  "email": "admin@mymquid.com",
  "role": "super_admin",
  "avatar": null
}
```

PUT /profile

Auth Required Update name and email

REQUEST BODY (UPDATEPROFILEDTO)

```
{
  "name": "Patrick Evra", // required
  "email": "admin@mymquid.com" // required
}
```

VALIDATION

name	min 2 chars, max 100
email	valid email, unique (409 if taken by another user)

RESPONSE 200 OK

Returns the updated AdminUser object.

ERROR RESPONSES

400	Validation error
409	Email already in use by another account

The frontend's Profile page uses React Hook Form + Zod validation and disables the Save button when there are no changes (isDirty === false).

PUT /profile/password

Auth Required Change password

REQUEST BODY (CHANGEPASSWORDDTO)

```
{
  "currentPassword": "old_password",
  "newPassword": "new_secure_password",
  "confirmPassword": "new_secure_password"
}
```

RESPONSES

200	{ "message": "Password updated" }
400	newPassword and confirmPassword don't match
401	currentPassword is incorrect

Always verify `currentPassword` against the stored hash (bcrypt) before updating. Never allow password change without confirming the current password.



08 — Frontend Integration Notes

How the frontend will connect to the real backend — the single swap point

Mock-to-Real API Swap Point

The entire mock API lives in a single file: `src/admin/mock/api.ts`. When the backend is ready, replace the function bodies in that file with Axios calls. No other files need to change — all frontend components import from this file.

SUGGESTED AXIOS INSTANCE (CREATE: SRC/LIB/AXIOS.TS)

```
import axios from "axios";

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || "http://localhost:3000/api/v1",
  headers: { "Content-Type": "application/json" },
});

api.interceptors.request.use((config) => {
  const auth = JSON.parse(
    localStorage.getItem("mymquid-admin-auth") || "{}"
  );
  const token = auth?.state?.token;
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

api.interceptors.response.use(
  (res) => res,
  (err) => {
    if (err.response?.status === 401) {
      localStorage.removeItem("mymquid-admin-auth");
      window.location.href = "/admin/login";
    }
    return Promise.reject(err);
  }
);

export default api;
```

MyMquid API Documentation - Confidential

The frontend currently hardcodes categories in `src/admin/mock/data.ts`. Add a categories endpoint or seed these to the database:

```
[
  "Company News",
  "Solutions",
  "Insights",
  "Case Studies"
]
```

Optionally add GET `/blog/categories` returning this list and wire it to the frontend's category filter dropdown.

ASPECT	PUBLIC	ADMIN
Endpoint	<code>/blog/public</code>	<code>/blog</code>
Auth	None	JWT required
Status filter	published only	all statuses
Used by	<code>src/pages/Blog.tsx</code>	<code>src/admin/</code>

The public blog page (`/blog`) calls the same mock API as the admin. When connecting, it should use an unauthenticated endpoint.

Environment Variables (Frontend)

ADD TO .ENV.LOCAL

```
VITE_API_URL=http://localhost:3000/api/v1
```

NESTJS CORS CONFIGURATION

```
app.enableCors({
  origin: [
    "http://localhost:5173", // Vite dev server
    "https://mymquid.com",  // production
  ],
  credentials: true,
});
```

Quick Reference: All Endpoints

METHOD	PATH	AUTH	ROLE	DESCRIPTION
POST	<code>/auth/login</code>	—	—	Login, get JWT
POST	<code>/auth/logout</code>	✓	any	Logout, invalidate token
POST	<code>/auth/forgot-password</code>	—	—	Send reset email
POST	<code>/auth/reset-password</code>	—	—	Reset password via token
GET	<code>/auth/me</code>	✓	any	Get own user from JWT
GET	<code>/blog</code>	✓	any	List posts (filterable, paginated)
POST	<code>/blog</code>	✓	any	Create new post

METHOD	PATH	AUTH	ROLE	DESCRIPTION
GET	/blog/public	—	—	Public list (published only)
GET	/blog/:id	✓	any	Get single post by ID
PUT	/blog/:id	✓	any*	Update post (* staff: own posts only)
DELETE	/blog/:id	✓	super_admin	Delete post
GET	/dashboard/stats	✓	any	Post count stats
GET	/dashboard/activity	✓	any	Recent activity feed
GET	/dashboard/chart	✓	any	Posts-per-day chart data
GET	/notifications	✓	any	User's notifications
PATCH	/notifications/read-all	✓	any	Mark all as read
PATCH	/notifications/:id/read	✓	any	Mark one as read
GET	/profile	✓	any	Get own profile
PUT	/profile	✓	any	Update name and email
PUT	/profile/password	✓	any	Change password